# BTEUP Diploma CSE — 5th Semester

# MOBILE APPLICATION DEVELOPMENT

Complete Syllabus-Oriented 5-Mark Notes

Easy Hinglish • Technical keywords in English • Diagrams / examples where useful

> Exam writing formula: Definition + labelled diagram/flow (where useful) + 5–7 explained points + example + short conclusion.

Important: These notes are a structured study aid based on the syllabus topics available in this conversation. They are not a guarantee of exact exam questions or marks. Verify the final topic list against your official syllabus sheet.

## How to use these notes

1. First learn the definition and headings; then practise explaining each point in your own words.

2. Draw diagrams neatly for architecture, lifecycle, provider flow and application workflow.

3. For code questions, understand what each line does; exact memorisation is less important than correct logic.

4. Similar questions are cross-referenced rather than repeated, but write the full relevant explanation in the exam.

5. Android tooling changes over time: Eclipse + ADT is legacy; Android Studio is the current recommended IDE.

## Contents / syllabus coverage

Unit I: Android platform, architecture, SDK, setup, first application, app anatomy, Manifest

Unit II: Activities/lifecycle, Context, Services, Intents, broadcasts, filters, permissions, Manifest settings

Unit III: UI elements, layouts, drawing, animation

Unit IV: Testing, publishing, Preferences, resources and hierarchy

Unit V: Storage APIs, SQLite, Content Providers, data sharing, networking/Web APIs, Telephony, deployment

Additional revision: Android terminology, common comparisons, quick exam checklist

# UNIT I — INTRODUCTION TO ANDROID

## Q1. Explain Android architecture with a neat diagram. (5 marks)

Definition / Introduction: Android is a Linux-based software platform for mobile and other smart devices. Its layered architecture separates application services, runtime, libraries and hardware-level operations.

Diagram / Flow:
Applications
↓
Application Framework
↓
Android Runtime (ART) + Native Libraries
↓
Hardware Abstraction Layer (HAL)
↓
Linux Kernel

Main points:

1. Applications layer: Contains system and user apps such as Phone, Camera, Contacts and installed applications.

2. Application Framework: Provides APIs and services. Activity Manager handles activity lifecycle; Notification Manager manages notifications; Resource Manager provides access to app resources.

3. Android Runtime (ART): Executes application code and provides core libraries. Modern Android primarily uses ART.

4. Native Libraries: Libraries such as SQLite, graphics and media libraries support databases, rendering and multimedia.

5. Hardware Abstraction Layer (HAL): Provides standard interfaces between Android framework services and hardware-specific implementations such as camera and audio.

6. Linux Kernel: Handles device drivers, memory/process management, networking, power management and core security.

Conclusion: Layered architecture makes Android modular and gives applications standard APIs to use device features.

## Q2. What is AndroidManifest.xml? Explain its elements and uses. (5 marks)

Definition / Introduction: AndroidManifest.xml is the application's configuration file. It declares components, permissions, features and other information that Android needs before running the app.

Main points:

1. : Root element containing the app manifest declarations.

2. : Holds application-level settings such as label, icon, theme and component declarations.

3. : Declares a screen component.

4. : Declares a service component.

5. : Declares a broadcast receiver.

6. : Declares a content provider.

7. : Requests access to protected capabilities, such as Internet or camera.

8. : Declares which implicit intent actions/data a component can handle.

Example: A launcher activity commonly has an intent filter with action `MAIN` and category `LAUNCHER`. If that activity has an intent filter, modern Android requires an explicit `android:exported` value.

Conclusion: The Manifest connects the app's components and configuration with the Android system.

## Q3. Explain Android SDK and steps to create a simple Android application. (5 marks)

Definition / Introduction: Android SDK (Software Development Kit) contains platform APIs and tools used to build, test, debug and package Android applications.

Main points:

1. SDK Platforms: APIs and libraries for a selected Android version.

2. Build Tools: Compile, package and prepare application builds.

3. Platform Tools / ADB: Communicate with connected devices and help with debugging.

4. Emulator / AVD: Run a virtual Android device on a computer.

5. SDK Manager: Install and update SDK platforms and tools.

6. Project creation: In Android Studio, create a project, select a template, name and language, then build the UI and application logic.

7. Run and debug: Test on an emulator or a real device and correct errors.

Example: A simple first app can display “Hello Android” using a TextView in its layout.

Conclusion: The SDK supplies the essential environment for Android development and testing.

## Q4. Explain the anatomy of an Android application. (5 marks)

Definition / Introduction: An Android app consists of application components, source code, resources and configuration files that work together.

Diagram / Flow:

```
Android App
├─ Activities / Services / Receivers / Providers
├─ Source code
├─ AndroidManifest.xml
└─ res/ (layouts, images, strings, styles)
```

Main points:

1. Activity: Represents a user-facing screen, such as Login or Home.

2. Service: Supports work that may continue without a visible UI, subject to Android background-execution rules.

3. Broadcast Receiver: Responds to supported broadcasts from the system or applications.

4. Content Provider: Offers a standard interface for managing and sharing structured data.

5. Manifest: Declares components, permissions and configuration.

6. Resources: Store layouts, strings, images, colours, dimensions and styles separately from source code.

7. Source code: Java/Kotlin classes implement app behaviour.

Conclusion: These parts separate UI, behaviour, data and configuration, making the app easier to maintain.

## Q5. Explain Android installation and Eclipse setup. (5 marks)

Definition / Introduction: Eclipse was historically used for Android development with the Android Development Tools (ADT) plugin. It is now a legacy setup; Android Studio is the recommended modern IDE.

Main points:

1. Legacy prerequisites: Install a compatible JDK and Eclipse version.

2. Android SDK: Download the SDK and install required platform packages.

3. ADT plugin: Configure the compatible ADT plugin in Eclipse.

4. SDK path: Set the SDK location and install platform tools.

5. Virtual device: Create an Android Virtual Device (AVD) to run the app.

6. Create and test: Create a project, write code, build it and run on an emulator or device.

7. Modern approach: Install Android Studio and follow its setup wizard to configure SDK components, then create and run a project.

Conclusion: Eclipse + ADT is mainly of historical importance; use Android Studio for current projects.

# UNIT II — APPLICATION DESIGN ESSENTIALS

## Q6. What is an Activity? Explain its lifecycle. (5 marks)

Definition / Introduction: An Activity is an Android component that represents a user-interaction screen. Its lifecycle callbacks tell the app when the screen is created, shown, paused, stopped or destroyed.

Diagram / Flow:
onCreate() → onStart() → onResume() → Running
Running → onPause() → onStop() → onDestroy()
Stopped → onRestart() → onStart()

Main points:

1. onCreate(): Initialises the activity and its UI.

2. onStart(): Makes the activity visible.

3. onResume(): Brings it to the foreground for user interaction.

4. onPause(): Called when the activity loses foreground focus; pause sensitive or ongoing UI work.

5. onStop(): Called when the activity is no longer visible.

6. onRestart(): Called when a stopped activity is about to start again.

7. onDestroy(): Called before the activity is destroyed; it is not guaranteed if the system kills the process.

Conclusion: Lifecycle callbacks help manage state and resources as users move between screens.

## Q7. What is an Intent? Explain explicit and implicit intents. (5 marks)

Definition / Introduction: An Intent is a messaging object used to request an action from another Android component, such as starting an activity, starting a service or sending a broadcast.

Main points:

1. Explicit Intent: Names the destination component; commonly used to open another activity in the same app.

2. Implicit Intent: Specifies an action or data without naming the exact component; Android resolves a suitable handler.

3. Extras: Intent can carry additional key-value data between components.

4. Action and data: Implicit intents can specify actions such as viewing a web page or sharing text.

5. Resolution: If no component can handle an implicit intent, the app should handle the possible failure.

Example: Explicit: `new Intent(MainActivity.this, SecondActivity.class)`. Implicit: create an `ACTION_VIEW` intent with a website URI, then call `startActivity()`.

Conclusion: Intents enable communication between Android components and apps.

## Q8. Explain Android Services and their types. (5 marks)

Definition / Introduction: A Service is an Android component for operations that do not require a user interface. Services are subject to platform background-execution rules.

Main points:

1. Started Service: Started by a component and continues until stopped or terminated.

2. Bound Service: Allows another component to bind and interact through an interface.

3. Foreground Service: Performs user-noticeable ongoing work and must show a notification, with applicable permissions and restrictions.

4. onCreate(): Initialises the service.

5. onStartCommand(): Handles a start request for a started service.

6. onBind(): Returns the interface used by bound clients.

7. onDestroy(): Performs cleanup before service destruction.

Example: Music playback is a common foreground-service use case when the playback needs to continue visibly while the user uses another app.

Conclusion: Services support ongoing or component-shared work without providing a screen of their own.

## Q9. Explain Broadcast Receivers and broadcasting intents. (5 marks)

Definition / Introduction: A Broadcast Receiver responds to broadcast messages sent by the Android system or an application.

Main points:

1. Broadcast Intent: Carries an action and optional data to notify interested components.

2. onReceive(): Callback invoked when a receiver receives a matching broadcast.

3. Registration: A receiver may be registered in the Manifest or dynamically using context APIs, subject to Android version restrictions.

4. Short processing: onReceive() should finish quickly; use an appropriate scheduled/background-work API for longer tasks.

5. System restrictions: Some system broadcasts cannot be received through manifest-declared receivers by ordinary apps.

6. App broadcasts: Use suitable action names and restrict access where appropriate.

Example: A simplified app broadcast can be sent with `sendBroadcast(new Intent("com.example.MY_EVENT"))`; a registered receiver checks the action in `onReceive()`.

Conclusion: Broadcasts provide event-based communication between components.

## Q10. Explain Intent Filters and their elements. (5 marks)

Definition / Introduction: An Intent Filter declares which implicit intents a component can handle. It is commonly placed in the Manifest.

Main points:

1. Action: Names the operation, such as MAIN or VIEW.

2. Category: Adds a category requirement, such as LAUNCHER.

3. Data: Can specify URI scheme, host, path or MIME type.

4. Matching: Android compares intent properties with filters to resolve a handler.

5. Launcher entry: MAIN action with LAUNCHER category makes an activity a launcher entry.

6. Explicit intents: Directly target a named component and do not rely on filter matching in the same way.

Example: A launcher activity uses an `intent-filter` containing `android.intent.action.MAIN` and `android.intent.category.LAUNCHER`.

Conclusion: Intent filters allow Android to discover components able to handle a given action.

## Q11. Explain Android Permissions and their types. (5 marks)

Definition / Introduction: Permissions protect sensitive data and device features by controlling what an application can access.

Main points:

1. Normal permissions: Low-risk operations; usually granted automatically, e.g. INTERNET.

2. Dangerous permissions: Sensitive data or features; require runtime approval where applicable, e.g. location or camera.

3. Signature permissions: Generally granted to apps signed with the same certificate as the declaring app.

4. Special access: Some capabilities require separate approval in system settings.

5. Manifest declaration: Declare required permissions with uses-permission.

6. Least privilege: Request only permissions required for the feature and explain their use to users.

Example: Manifest example: `<uses-permission android:name="android.permission.INTERNET" />`. A camera feature may also require a runtime permission request.

Conclusion: Permissions protect user privacy and limit unauthorised access.

## Q12. Explain Application Context and its uses. (5 marks)

Definition / Introduction: Context provides access to application environment information, resources and system services.

Main points:

1. Activity Context: Associated with an Activity; suitable for UI operations and Activity-related tasks.

2. Application Context: Associated with the application lifetime; suitable for app-wide operations.

3. Resources: Access strings, assets and other resources.

4. Preferences: Access app settings such as SharedPreferences.

5. System services: Obtain supported services through context APIs.

6. Memory safety: Avoid storing an Activity Context in a long-lived object because it can cause memory leaks.

Example: Use `getApplicationContext()` when an application-wide context is appropriate, such as for some long-lived helpers.

Conclusion: Context connects app code to Android resources and services.

## Q13. Explain common settings in AndroidManifest.xml. (5 marks)

Definition / Introduction: The Manifest records app-level settings and declarations required by the Android system.

Main points:

1. Label and icon: Set the display name and launcher icon.

2. Theme: Apply app-wide or component-level visual styling.

3. Component declarations: Declare activities, services, receivers and providers.

4. Permissions: Request access to protected capabilities.

5. Features: Declare hardware/software features using uses-feature when relevant.

6. Launcher configuration: Use MAIN action and LAUNCHER category for the launcher activity.

7. Exported status: For components with intent filters, modern Android requires an explicit exported setting; exported components may be launchable by other apps.

Conclusion: Manifest settings control app configuration, component discovery and access boundaries.

# UNIT III — USER INTERFACE DESIGN

## Q14. Explain Android UI components with examples. (5 marks)

Definition / Introduction: UI components are visual elements that display information or let users interact with an application.

Main points:

1. TextView: Displays text.
2. EditText: Accepts text input.
3. Button: Triggers an action when clicked.
4. ImageView: Displays images or drawable resources.
5. CheckBox: Allows independent selection of options.
6. RadioButton: Used for selecting among related alternatives, often inside RadioGroup.
7. Switch: Represents an on/off setting.
8. ProgressBar: Shows progress or loading state.

Example: A login screen may use TextView for labels, EditText for username/password and Button for submitting the form.

Conclusion: UI widgets make an app interactive and understandable.

## Q15. Explain Android Layouts and their types. (5 marks)

Definition / Introduction: A layout is a ViewGroup that arranges child views on the screen.

Main points:

1. LinearLayout: Arranges children vertically or horizontally.
2. ConstraintLayout: Positions views using constraints; useful for responsive, complex screens.
3. FrameLayout: Places views in a frame, allowing stacking or overlays.
4. TableLayout: Arranges content in rows, often using TableRow.
5. RelativeLayout: Positions views relative to parent or other views; common in older designs.
6. layout_width / layout_height: Control view dimensions.
7. padding / margin: Control inner spacing and outer spacing respectively.

Example: A vertical LinearLayout can place a TextView above a Button.

Conclusion: Choosing a suitable layout keeps the UI organised across screen sizes.

## Q16. Explain drawing in Android using Canvas and Paint. (5 marks)

Definition / Introduction: Android custom drawing can be implemented by creating a custom View and drawing on its Canvas using a Paint object.

Main points:

1. Canvas: Provides the drawing surface.
2. Paint: Sets colour, stroke width, text size and fill/stroke style.
3. Custom View: A class extending View can implement custom graphics.
4. onDraw(): Override this callback to draw when Android renders the view.
5. Drawing methods: Methods include drawCircle(), drawRect(), drawLine() and drawText().
6. Performance: Avoid heavy operations inside onDraw(); invalidate the view when it needs redrawing.

Example: A custom View can create a Paint, set its colour and call `canvas.drawCircle(150, 150, 60, paint)`.

Conclusion: Canvas and Paint enable charts, shapes and custom visual elements.

## Q17. Explain animation in Android and its types. (5 marks)

Definition / Introduction: Animation changes visual appearance or object properties over time to provide movement and feedback.

Main points:

1. View/Tween animation: Applies alpha, scale, translate or rotate transformations to a view.

2. Frame animation: Displays a sequence of drawable frames.

3. Property animation: Changes actual object properties over time, e.g. alpha or translationX.

4. Duration: Controls how long an animation takes.

5. Interpolator: Controls the rate of change over time.

6. Use and caution: Use animations for feedback and transitions, but avoid excessive motion or performance-heavy effects.

Example: An ObjectAnimator can animate a view’s alpha from 0 to 1 over 1000 milliseconds.

Conclusion: Animations can make interfaces smoother and more engaging when used appropriately.

# UNIT IV — TESTING, PUBLISHING AND RESOURCES

## Q18. Explain testing of Android applications and its types. (5 marks)

Definition / Introduction: Testing checks whether an app meets requirements and works correctly across expected conditions.

Main points:

1. Unit testing: Tests an individual method or class.
2. Integration testing: Tests interaction between components.
3. Functional testing: Checks whether features produce expected results.
4. UI testing: Tests screen elements, navigation and user workflows.
5. Performance testing: Checks responsiveness, memory use and speed.
6. Compatibility testing: Checks behaviour on different devices, Android versions and screen sizes.
7. Regression testing: Re-tests existing features after changes.

Conclusion: A structured testing process finds defects early and improves reliability.

## Q19. Explain the process of publishing an Android application. (5 marks)

Definition / Introduction: Publishing prepares a tested application for distribution to users through a store or another suitable channel.

Main points:

1. Final testing: Check important workflows, crashes and device compatibility.
2. Versioning: Set appropriate version name and version code.
3. Release build: Create a release build rather than a debug build.
4. Signing: Sign the release package and protect the signing key.
5. Package format: APK is installable; AAB is a publishing format from which Google Play can generate device-specific APKs.
6. Store listing: Prepare description, screenshots, icon and required details.
7. Policy review: Complete privacy, content and permission declarations; submit for review.
8. Maintenance: Monitor feedback and publish updates as needed.

Conclusion: Testing, signing and policy compliance are essential before public release.

## Q20. Explain Android Preferences and SharedPreferences. (5 marks)

Definition / Introduction: SharedPreferences is a traditional Android API for storing small amounts of key-value data, such as simple settings.

Main points:

1. Key-value model: Each stored value is associated with a key.
2. Supported values: Commonly supports strings, booleans and primitive numeric values.
3. Access: getSharedPreferences() opens a named preference file.
4. Editing: edit() and put methods prepare changes.
5. Persistence: apply() saves changes asynchronously; get methods read stored values.
6. Security: Do not store sensitive secrets as plain preferences; choose appropriate secure storage.

Example: Example: `prefs.edit().putBoolean("darkMode", true).apply();` then read using `prefs.getBoolean("darkMode", false)`.

Conclusion: SharedPreferences is suitable for small app settings, not complex relational records.

## Q21. Explain Android Resources and resource hierarchy. (5 marks)

Definition / Introduction: Resources are non-code assets such as layouts, images, strings and colours. Android selects the best matching resource based on device configuration.

Main points:

1. layout/: Contains UI layout XML files.

2. drawable/: Contains images and drawable XML.

3. mipmap/: Commonly stores launcher icons.

4. values/: Contains strings, colours, dimensions, styles and themes.

5. menu/ and xml/: Contain menu definitions and other XML configuration.

6. raw/ and anim/: Store raw files and animation resources.

7. Qualifiers: Folders such as values-hi, drawable-hdpi and layout-land target language, density or orientation.

Example: Define a string in `res/values/strings.xml` and access it with `getString(R.string.app_name)`.

Conclusion: Resources separate content from code and help support localisation and different devices.

## Q22. Explain different types of Android resources. (5 marks)

Definition / Introduction: Android resources are external assets and values referenced through the generated R class.

Main points:

1. Layout resources: Define screen structure in XML.

2. Drawable resources: Images, shapes and visual backgrounds.

3. String resources: Labels and messages, supporting translation.

4. Colour resources: Reusable colours.

5. Dimension resources: Values for spacing and sizes, commonly dp/sp.

6. Style and theme resources: Define reusable UI appearance.

7. Menu resources: Define menu items.

8. Raw/XML/animation resources: Store raw files, configuration and animation definitions.

Example: A value declared as `<string name="welcome">Welcome</string>` can be accessed through `R.string.welcome`.

Conclusion: Resource files improve reuse, consistency and maintainability.

# UNIT V — ANDROID APIs AND DATA

## Q23. Explain Android Storage APIs and their types. (5 marks)

Definition / Introduction: Storage APIs allow an app to save and retrieve files, preferences and structured records.

Main points:

1. SharedPreferences: Stores small key-value settings.

2. Internal storage: Stores app-private files, generally inaccessible directly to other apps.

3. App-specific external storage: Stores app files in a shared-storage area with platform-specific access rules.

4. SQLite: Stores structured relational data in local tables.

5. Content Providers: Offer standard access to structured data that may be shared.

6. Shared documents/media: Use suitable APIs such as Storage Access Framework or media APIs for user-visible files.

7. Selection: Choose storage based on data structure, privacy, size and sharing requirements.

Conclusion: The correct storage API depends on whether data is private, structured, or intended to be shared.

## Q24. Explain SQLite database in Android. (5 marks)

Definition / Introduction: SQLite is a lightweight embedded relational database used to store structured data locally on an Android device.

Main points:

1. Tables: Organise data into rows and columns.

2. SQL operations: Support INSERT, SELECT, UPDATE and DELETE.

3. Offline access: Local data can be accessed without an Internet connection.

4. SQLiteOpenHelper: Traditional helper for database creation and version management.

5. onCreate(): Creates tables when the database is first created.

6. onUpgrade(): Handles schema changes between database versions.

7. Database access: getWritableDatabase() and getReadableDatabase() open database access.

Example: A Student table can be created with `CREATE TABLE Student(id INTEGER PRIMARY KEY, name TEXT)`. Use parameterised queries or ContentValues for safer data operations.

Conclusion: SQLite is useful for local structured data such as student records or offline inventory.

## Q25. Explain Content Providers in Android. (5 marks)

Definition / Introduction: A Content Provider manages structured data and exposes a standard URI-based interface for authorised access.

> Diagram / Flow:
> Client App → ContentResolver → Content URI → Content Provider → Stored Data

Main points:

1. Content Provider: Implements data access operations.

2. Content URI: Identifies the data, commonly using the content:// scheme.

3. ContentResolver: Client-side interface used to communicate with a provider.

4. query(): Reads records, often returned through a Cursor.

5. insert(), update(), delete(): Create, modify or remove records.

6. Permissions: Provider permissions or URI grants can restrict access.

7. Abstraction: The provider can hide the underlying database implementation.

Example: An authorised app can access Contacts Provider data when the necessary permissions and platform conditions are satisfied.

Conclusion: Content Providers support standardised and controlled data access between applications.

## Q26. Explain sharing data between Android applications. (5 marks)

Definition / Introduction: Android provides several mechanisms for transferring information between apps, depending on whether the data is a simple action, a file or structured records.

Main points:

1. Explicit Intent: Targets a known component.

2. Implicit Intent: Requests an action such as viewing or sharing content.

3. Content Provider: Exposes structured records through a standard interface.

4. ContentResolver: Lets a client query or modify provider data when authorised.

5. FileProvider: Shares files through secure content URIs and temporary URI permissions.

6. Network APIs: Allow apps to exchange data through a server when appropriate.

7. Security: Avoid exposing raw file paths or sharing sensitive data without authorisation.

Example: For text sharing, create an ACTION_SEND intent, set MIME type to text/plain, attach EXTRA_TEXT and launch an Intent.createChooser().

Conclusion: Select the sharing mechanism according to data type and access requirements.

## Q27. Explain Android Networking APIs and Web APIs. (5 marks)

Definition / Introduction: Networking APIs let apps communicate with remote servers. Web APIs expose server functions or data over HTTP/HTTPS.

> Diagram / Flow:
> Android App → HTTP/HTTPS Request → Web Server/API → JSON Response → App

Main points:

1. Internet permission: Declare android.permission.INTERNET for network access.

2. HTTP/HTTPS: Protocols for client-server requests; HTTPS encrypts data in transit.

3. REST methods: GET reads data, POST submits/creates, PUT updates/replaces and DELETE removes a resource, according to the API design.

4. JSON: Common format for structured request and response data.

5. Libraries: HttpURLConnection, OkHttp and Retrofit can be used for network calls.

6. Background execution: Do not block the main UI thread with network requests.

7. Error handling: Handle timeouts, connection failures, server errors and invalid responses.

Conclusion: Networking APIs connect Android apps to online services, remote databases and web backends.

## Q28. Explain Android Telephony APIs. (5 marks)

Definition / Introduction: Telephony APIs provide access to supported mobile-network, SIM and call-state information under Android's privacy and permission rules.

Main points:

1. TelephonyManager: Provides supported telephony and network information.

2. Network type: Can report supported data-network type when available.

3. SIM/operator information: Some operator and SIM details may be accessible, subject to permissions and restrictions.

4. Call state: Supported APIs can provide call-state information under applicable rules.

5. SMS-related features: May require permissions, default SMS app status or other policy compliance.

6. Device support: Wi-Fi-only devices may not provide telephony services.

7. Privacy: Apps cannot freely access every phone number, call detail or SMS record.

Example: A TelephonyManager can be obtained from Context using `getSystemService(TELEPHONY_SERVICE)`; check device support and required access before calling restricted APIs.

Conclusion: Telephony APIs enable network-aware features while respecting Android privacy controls.

## Q29. Explain deployment of Android applications. (5 marks)

Definition / Introduction: Deployment is the process of preparing an app for installation and distributing it to users or target devices.

Main points:

1. Prepare release: Finish features, test workflows and resolve critical bugs.

2. Versioning: Set version name and version code.

3. Build: Generate a signed release APK or an AAB for Play publishing.

4. Signing key: Keep the signing key and credentials secure.

5. Distribution: Distribute via Google Play, direct APK or an approved private channel.

6. Policy compliance: Complete privacy, permissions and store requirements.

7. Release and monitor: Publish, monitor crashes/feedback and provide updates.

Conclusion: A controlled deployment process improves reliability, security and update management.

# EXTRA REVISION — COMPARISONS AND EXAM CHECKLIST

### Q30. Differentiate between Activity and Service. (5 marks)

Definition / Introduction: Activity and Service are Android components with different roles.

Main points:

1. Purpose: Activity represents a user-facing screen; Service supports operations without its own UI.
2. Interaction: Activity handles direct user interaction; Service performs ongoing or bound work.
3. Lifecycle: Activity has screen lifecycle callbacks; Service has service-specific callbacks.
4. Visibility: Activity is associated with a visible screen; a Service may work without a visible screen.
5. Example: Login screen is an Activity; music playback may use a foreground Service.

Conclusion: Use Activity for UI and an appropriate service/work API for background-related operations.

### Q31. Differentiate between SQLite and SharedPreferences. (5 marks)

Definition / Introduction: SQLite and SharedPreferences store local app data but suit different data needs.

Main points:

1. Data model: SQLite uses relational tables; SharedPreferences uses key-value pairs.
2. Complexity: SQLite supports structured queries; SharedPreferences suits simple settings.
3. Data examples: SQLite: students/products; preferences: theme choice or a simple flag.
4. Operations: SQLite supports SQL CRUD operations; preferences use keys and values.
5. Selection: Use the simplest appropriate storage mechanism for the data.

Conclusion: Use SQLite for structured records and SharedPreferences for small settings.

### Q32. Differentiate between APK and AAB. (5 marks)

Definition / Introduction: APK and AAB are Android application packaging formats, but they serve different purposes.

Main points:

1. APK: Installable package format for Android devices.
2. AAB: Publishing format containing compiled code and resources from which a store can generate APKs.
3. Distribution: APKs can be directly distributed; AAB is commonly uploaded to Google Play.
4. Device optimisation: AAB lets Google Play deliver device-appropriate APKs.
5. Signing: Release packages and publishing workflows must follow appropriate signing requirements.

Conclusion: APK is an installable package, while AAB is primarily a publishing format.

## Final exam checklist

- Revise all definitions and technical keywords.
- Practise Android architecture and Activity lifecycle diagrams.
- Practise Intent, Manifest, SQLite and Content Provider examples.
- Learn the differences: explicit/implicit intent, Activity/Service, SQLite/Preferences, APK/AAB.

• For each 5-mark answer, write an introduction, 5–7 explained points, an example/diagram where relevant, and a short conclusion.

• Compare these notes against the official syllabus sheet before the exam; no selected-question list can guarantee the exact paper.